# Assignment 2: Robo-Football

## Overview

You will build a two-player, turn-based tactical "robo-football" game from scratch, then design and evaluate agents that play it using adversarial search. This assignment has two parts: **(1) build the game, (2) build and analyse players for it**.

Why **Robo**-football? Because all the players in a team are *think* as one, e.g. as if there is a common controller deciding what each player should do at each moment (which is not the case in real, human football!).

The rules below are the complete and exact specification; if a situation isn't covered by them, you must make a reasonable, documented design decision.

## Game Rules

**Board.** A grid of 6 rows (0–5) and 5 columns (0–4). Team A defends row 0 and attacks row 5. Team B defends row 5 and attacks row 0.

**Pieces.** Each team has 4 outfield players and 1 goalkeeper. At kickoff, teams line up in a fixed starting formation of your own design (document it), mirrored for both teams. No two pieces may ever occupy the same cell.

**Turn order.** Players strictly alternate turns, one team per turn, regardless of which team has the ball: Team A moves, then Team B, then Team A, and so on. Which team acts next depends only on whose turn it is — not on possession.

**If the team whose turn it is has the ball**, it must choose exactly one of the following for its ball-carrying player:

- **Dribble** — move the ball carrier one cell in any of the 8 directions, into an empty cell. If, after this move, an opposing player is adjacent (including diagonally) to the carrier's new cell, that opponent

immediately wins the ball (possession switches to them). If more than one opposing player is adjacent, break the tie by any fixed, consistent rule you define and document.

- **Pass** — send the ball to a teammate, but only if the straight line between the carrier and that teammate (horizontal, vertical, or exact diagonal) is not blocked by any opposing player standing on it. A legal pass always succeeds.
- **Shoot** — only allowed if the ball carrier is in the attacking third of the board (the two rows nearest the opponent's goal). The shot is a goal unless the opposing goalkeeper is within 1 column of the shooter's column, in which case it is saved and the goalkeeper gains the ball.

**If the team whose turn it is does not have the ball**, it must move exactly one of its own players one cell in any of the 8 directions, into an empty cell (a defensive repositioning move). The goalkeeper is restricted to moving sideways only, and must stay on its own goal line.

**Goals.** When a goal is scored, all players reset to the kickoff formation, and the team that conceded restarts with the ball. Play continues — the match does not end when a goal is scored.

**End of match.** The match ends after a fixed number of total turns (you choose this budget, e.g. 60–100; document and justify your choice). The team with more goals wins; equal goals is a draw.

# Part 1 — Game Environment

Implement the game exactly as specified above: state representation, legal move generation, the rule for applying a move to produce a new state, and detection of the end of the match. Your environment must be usable by two independent agents without either one needing to know how the other is implemented.

Include a way to print or visualize the board state, and enough automated checks to convince yourself that the rules are correctly enforced — e.g. no illegal overlaps, no illegal shots, correct tackle and save resolution.

# Part 2 — Players and Search

Implement and compare different agents e.g.:

1. **Random** — picks uniformly among legal moves. Your baseline.
2. **Greedy, no search** — picks the single move that looks best under one of your heuristic evaluation functions, with no lookahead at all.
3. **Minimax, depth-limited, no pruning.**
4. **Minimax with alpha-beta pruning**, same depth and same heuristic as (3), for direct comparison.
5. **At least two distinct heuristic evaluation functions**, used with your alpha-beta agent, that reflect different ideas about what matters in this game (e.g. one focused on field position, one focused on passing/defensive structure) — you decide what to include and why.
6. **Monte-carlo tree search with full/depth-limited roll-outs.

# Part 3 — Experiments and Analysis

Run the experiments below and report your results (tables/plots plus written interpretation — numbers alone are not sufficient):

- **Pruning vs. no pruning:** for the same heuristic and at least 3 different search depths, report nodes expanded and wall-clock time with and without alpha-beta pruning, on at least two different board positions (not just the kickoff position).
- **Greedy vs. search:** run a tournament (enough games to be statistically meaningful, e.g. 20+ per pairing, alternating which side each agent plays) between your greedy agent and your alpha-beta agent. Report win/loss/draw rates.
- **Heuristic vs. heuristic:** run a tournament between your two (or more) heuristics under otherwise identical search settings. Report results and explain, in terms of the game, why one performed better (or comment if they seem evenly matched).
- **Depth vs. performance:** show how agent strength and computation cost change as you increase search depth, and discuss the trade-off.
- **MCTS vs other strategies** evaluate cost-benefit of using MCTS, etc.

The grading will be based on an **in-class exam** based on this assignment. You should expect to: hand-trace a few turns of minimax with alpha-beta pruning on a new position, explain and justify choices in your own heuristic, and interpret your own experimental results (e.g. explain why pruning saved what it saved on a specific position). Work you cannot explain in your own words will not receive credit, regardless of whether it runs correctly.